

Bypassing Low Type Filter in .NET Remoting

 tiraniddo.dev/2019/10/bypassing-low-type-filter-in-net.html

I recently added a new feature my [.NET remoting exploitation tool](#) which in many cases allows you to exploit an arbitrary service through serialization. This feature has always existed in the tool, if you passed the *useser* option, however it only worked if the service had enabled *Full Type Filter* mode, the default for remoting services is *Low Type Filter* which my tool couldn't easily exploit. I'm going to explain how I bypassed it *Low Type Filter* mode in the latest tool.

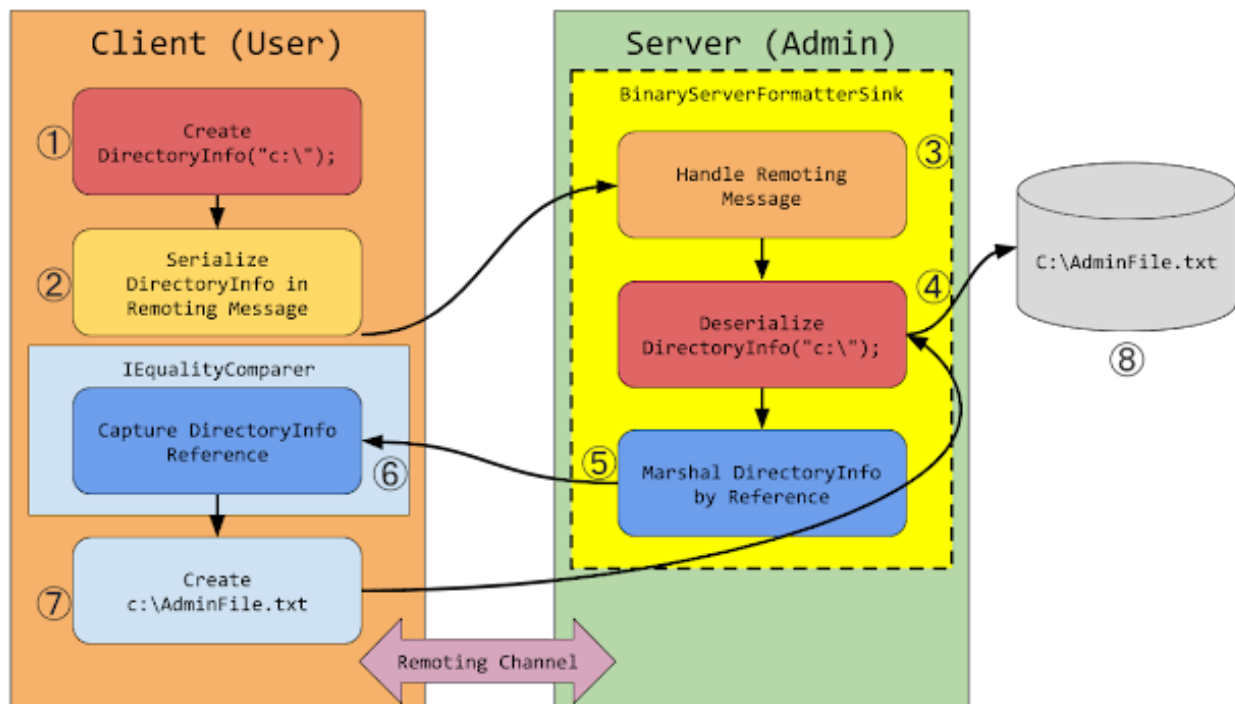
It's worth noting that this technique is currently unpatched, however no one should be using .NET remoting in a modern context (*cough* Visual Studio *cough*).

I'd recommend starting by reading my [previous blog post](#) on this subject as it describes where the Type Filtering comes into play. You can also read [this MSDN page](#) which describes what can and cannot be deserialized during a .NET remoting call with *Low* versus *Full Type Filtering* enabled.

In simple terms enabling *Low* (which is the default) over *Full* results in the following restrictions:

- Object types derived from *MarshalByRefObject*, *DelegateSerializationHolder*, *ObjRef*, *IEnvoyInfo* and *ISponsor* can not be deserialized.
- All objects which are deserialized must not Demand any CAS permission other than *SerializationFormatter* permission.

The *useser* technique abuses the fact that certain classes such as [DirectoryInfo](#) and [FileInfo](#) are both derived from *MarshalByRefObject* (MBR) and are also serializable. By deserializing an instance of one of the special classes inside a carefully crafted *Hashtable*, with a MBR instance of *IEqualityComparer* you can get the server to pass back the instance. As this object is passed back over a remoting channel the *DirectoryInfo* or *FileInfo* objects are marshalled by reference and are stuck inside the server. We can now call methods on the returned object to read and write arbitrary files, which can be used to get full code execution in the server. I've summarized the main interactions in the following diagram:



Low Type Filter acts to modify the behavior of the *BinaryServerFormatterSink* block, which encapsulates blocks 3, 4 and 5. The change in behavior blocks the *useser* technique in three ways.

Firstly in order to get the instance of the special object passed back to the client we need to pass a MBR *IEqualityProvider*. This will be blocked during handling of the remoting message (3).

Secondly when deserializing an instance of *FileInfo* or *DirectoryInfo* (4) a Demand is made for a *FileIOPermission* for the path to access. As the permission Demand is made during deserialization it hits the restriction that only *SerializationFormatter* permissions are allowed.

Thirdly, even if the object is deserialized successfully we'll hit a final problem, calling the *IEqualityProvider* (5 and 6) over a remoting channel to pass back the reference requires setting up a new TCP or Named Pipe connection. Setting up the connection will also hit the limited permissions and again throw an exception causing the call to fail.

How can we work around the three issues? Let's first bypass the type checking which prevents MBR objects being deserialized. If you dig into the code you'll find the type checks are performed in the `ObjectReader::CheckSecurity` method, which is as follows:

```

internal void CheckSecurity(ParseRecord pr) {
    Type t = pr.PRdtType;
    if ((object)t != null){
        if(IsRemoting) {
            if (typeof(MarshalByRefObject).IsAssignableFrom(t))
                throw new ArgumentException();
            FormatterServices.CheckTypeSecurity(t, formatterEnums.FEsecurityLevel);
        }
    }
}

```

The important thing to note is that the checks are only made if the *IsRemoting* property is true. What determines the value of the property? Again we can just look in the [reference source](#):

```

private bool IsRemoting {
    get {
        return (bMethodCall || bMethodReturn);
    }
}

```

What sets *bMethodCall* or *bMethodReturn*? They're set by the *BinaryFormatter* when it encounters the special *MethodCall* or *MethodReturn* record types. It turns out that maybe for performance or security (unclear) the formatter can special case these object types when used in .NET remoting and only storing properties of these objects when serializing and reconstructing the method objects when deserializing.

However if you read my previous blog post you'll notice something, I was unmarshalling a *MBR* instance of an *IMessage*, and that didn't hit the checks. This was because as long as the top level record is not a *MethodCall* or *MethodReturn* record type then we can deserialize anything we like, that was easy to bypass. In theory we can just pass a serialized Hashtable as the top level object, it'll cause the remoting server code to fault when trying to call methods on the message object but by then it'd be too late. In fact this is exactly what the *useser* option does anyway, however it's the second security feature which really causes us problems trying to get it to work on *Low Type Filter*.

When handling an incoming request is enables a *PermitOnly* CAS grant over the deserialization process, which only allows *SerializationFormatter* permissions to be asserted. You can see it in action in the reference source [here](#), which I've copied below.

```

PermissionSet currentPermissionSet = null;
if (this.TypeFilterLevel != TypeFilterLevel.Full) {
    currentPermissionSet = new PermissionSet(PermissionState.None);
    currentPermissionSet.SetPermission(

```

```

        new SecurityPermission(
            SecurityPermissionFlag.SerializationFormatter));
    }

    try {
        if (currentPermissionSet != null)
            currentPermissionSet.PermitOnly();

        // Deserialize Request - Stream to IMessage
        requestMsg = CoreChannel.DeserializeBinaryRequestMessage(
            objectUri, requestStream, _strictBinding, this.TypeFilterLevel);
    }
    finally {
        if (currentPermissionSet != null)
            CodeAccessPermission.RevertPermitOnly();
    }
}

```

As we're passing the *Hashtable* containing the serialized object we want to capture as well as the MBR *IEqualityComparer* as the top level object all of our machinations will run during this *PermitOnly* grant, which as I've already noted will fail. If we could defer the deserialization, or at least any privileged operation until after the CAS grant is reverted we'd be able to exploit this trick, but how can we do that?

One way to defer code execution is to exploit object finalization. Basically when an object's resources are about to be reclaimed by the GC it'll call the object's finalizer. This call is made on a GC thread completely outside the deserialization process and so wouldn't be affected by the CAS *PermitOnly* grant. In fact abusing finalizers was something I pointed out in my original research on .NET serialization, a good example is the infamous TempFileCollection class.

I thought about trying to find a useful gadget to exploit this, however there were two problems. First the difficulty in finding a suitable object which is both serializable and has a useful finalizer defined and second, the call to the finalizer is non-deterministic as it's whenever the GC gets called. In theory the GC might never be called.

I decided to focus on a different approach based on a non-obvious observation. The *PermitOnly* security behaviors of *Low Type Filter* only apply when calling a method on a server object, not deserializing the return value. Therefore if I could find somewhere in the server which calls back to a MBR object I control then I can force the server to deserialize an arbitrary object. This object can be used to mount the attack as the deserialization would not occur under the *PermitOnly* CAS grant and I can use the same *Hashtable* trick to capture a *DirectoryInfo* or *FileInfo* object.

In theory you could find an exposed method on the server object to use for this callback, however I wanted my code to be generic and not require knowledge of the server object outside of the knowing the URI. Therefore it'd have to be a method we can call on the MBR or base *Object* class. An initial look only shows one candidate, the *Object::Equals* method which takes a single parameter. Unfortunately most of the time a server object won't override this method and the default just performs reference equality which doesn't call any methods on the passed object.

The only other candidates are the *InitializeLifetimeServer* or *GetLifetimeService* methods which return an MBR which implements the *ILease* interface. I'm not going to go into what this is used for (you can read up on it on *MSDN*) but what I noticed was the *ILease* interface has a *Register* method which takes an object which implements *ISponsor* interface. If you registered an MBR object in the client with the server's lifetime service then when the server wants to check if the object should be destroyed it'll call the *ISponsor::Renewal* method, which gives us our callback. While the method doesn't return an object, we can just throw an exception with the *Hashtable* inside and exploit the service. Victory?

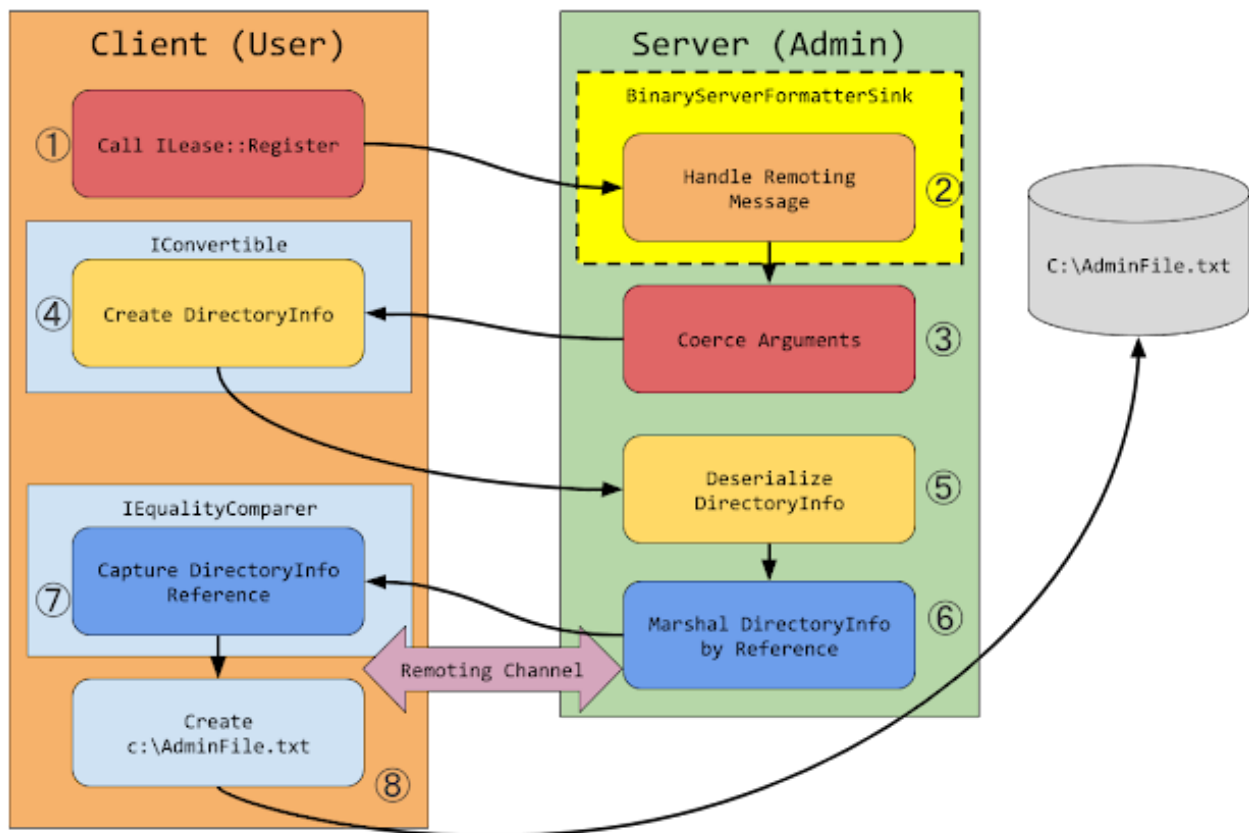
Not quite, it turns out that we've now got new problems. The first one is the *Renewal* call only happens when the lifetime counter expires, the default timeout is around 10 minutes from the last call to the server. This means that our exploit will only run at some long, potentially indeterminate point in time. Not the end of the world, but as frustrating as waiting for a GC run to get a finalizer executed. But the second problem seems more insurmountable, in order to set the *ISponsor* object we need to make an actual call to the server, however *Low Type Filter* would stop us from passing an MBR *ISponsor* object as the top level object would be a *MethodCall* record type which would throw an exception when it was encountered during argument deserialization.

What can we do? Turns out there's an easy way around this, the framework provides us with a full serializable *MethodCall* class. Instead of using the *MethodCall* record type we can instead package up a serializable *MethodCall* object as the top level object with all the data needed to make the call to *Register*. As the top level object is using a normal serialized object record type and not a *MethodCall* record type it'll never trigger the type checking and we can call *Register* with our MBR *ISponsor* object.

You might wonder if there's another problem here, won't deserializing the MBR cause the channel to be created and hit the *PermitOnly* CAS grant? Fortunately channel setup is deferred until a call is made on the object, therefore as long as no call is made to the MBR object during the deserialization process we'll be out of the CAS grant and able to setup the channel when the *Renewal* method is called.

We now have a way of exploiting the remoting service without knowledge of any specific methods on the server object, the only problem is we might need to wait 10 minutes to do it. Can we improve on the time? Digging further into default remoting implementation I noticed

that if an argument being passed to a method isn't directly of the required type the method `StackBuilderSink::SyncProcessMessage` will call `Message::CoerceArgs` to try the coerce the argument to the correct type. The fallback is to call `Convert::ChangeType` passing the needed type and the object passed from the client. To convert to the correct type the code will see if the passed object implements the `IConvertible` interface and call the `ToType` method on it. Therefore, instead of passing an implementation of `ISponsor` to `Register` we just pass one which implements `IConvertible` the remoting code will try and coerce it using `ChangeType` which will give us our needed callback immediately without waiting 10 minutes. I've summarized the attack in the following diagram:



This entire exploit is implemented behind the `use/lease` option. It works in the same way as `useser` but should work even if the server is running `Low Type Filter` mode. Of course there's caveats, this only works if the server sets up a bi-direction channel, if it registers a `TcpChannel` or `IpcChannel` then that should be fine, but if it just sets up a `TcpServerChannel` it might not work. Also you still need to know the URI of the server and bypass any authentication requirements.

If you want to try it out grab the code from [github](#) and compile it. First run the `ExampleRemotingServer` with the following command line:

```
ExampleRemotingService.exe -t low
```

This will run the example service with *Low Type Filter*. Now you can try *useser* with the following command line:

```
ExploitRemotingService.exe --useser tcp://127.0.0.1:12345/RemotingServer ls c:\
```

You should notice it fails. Now change *useser* to *uselease* and rerun the command:

```
ExploitRemotingService.exe --uselease tcp://127.0.0.1:12345/RemotingServer ls c:\
```

You should see a directory listing of the C: drive. Finally if you pass the *autodir* option the exploit tool will try and upload an assembly to the server's base directory and bootstrap a full server from which you can call other commands such as *exec*.

```
ExploitRemotingService.exe --uselease --autodir tcp://127.0.0.1:12345/RemotingServer exec  
notepad
```

If it all works you should find the example server will spawn notepad. This works on a fully up to date version of .NET (e.g. .NET 4.8).

The take away from this is DO NOT EVER USE .NET REMOTING IN PRODUCTION. Even if you're lucky and you're not exploitable for some reason the technologies should be completely deprecated and (presumably) will never be ported .NET Core.